

INNOVATION LAB MANAGEMENT

PROJETO AoR - 12ª EDIÇÃO

Conteúdo

1. ENQUADRAMENTO	4
2. AVALIAÇÃO	4
3. GRUPOS E ORIENTADORES	4
4. DOMÍNIO DO PROJETO	5
4.1. CONTEXTO	5
4.2. OBJETIVOS	5
4.3. GLOSSÁRIO DO DOMÍNIO	5
5. REQUISITOS FUNCIONAIS OBRIGATÓRIOS	6
5.1. PERFIS DE UTILIZADOR	6
5.1.1. Perfil de administrador	7
5.1.2. Perfil de gestor	7
5.1.3. Perfil de utilizador padrão	7
5.1.4. Perfil de utilizador não autenticado	7
5.2. ACESSO E AUTENTICAÇÃO	7
5.2.1. Registo	7
5.2.2. Autenticação	7
5.2.3. Terminar Sessão	7
5.2.4. Alterar palavra-passe	7
5.3. EXECUÇÃO DE SIMULAÇÕES	8
5.3.1. Monitorização Contínua	8
5.3.2. Representação Visual do Corpo Humano (2D)	8
5.3.3. Alertas	8
5.4. SUBMISSÃO DE LEITURAS FISIOLÓGICAS E CONSULTA DE ALERTAS	9
5.4.1. Submissão de Leituras (Batch)	9
5.4.2. Submissão de Leituras (Stream)	9
5.4.3. Consulta de Alertas	9
5.4.4. Consulta de Histórico de Alertas	9
5.5. CARREGAMENTO DE DADOS	9
5.5.1. Ficheiro com dados do cenário fisiológico clínico	9
5.5.2. Ficheiro com dados de alertas	10
5.6. RELATÓRIO POR AVALIAÇÃO	10
6. REQUISITOS FUNCIONAIS OPCIONAIS (BÓNUS)	11
6.1. MODO DEGRADADO (FORTEMENTE RECOMENDADO)	11
6.2. INTEGRAÇÃO COM O BIOGEARS (FORTEMENTE RECOMENDADO)	11
6.3. INTEROPERABILIDADE	11
6.4. FEATURE FLAGS	12
7. REQUISITOS TÉCNICOS	12
8. REQUISITOS DE QUALIDADE E SEGURANÇA	12
8.1. SESSION TIMEOUT	12
8.2. FORÇA DAS PALAVRAS-PASSE	12
8.3. ARMAZENAMENTO DE PALAVRAS-PASSE	12
8.4. CONTROLO DE ACESSO	12
8.5. VALIDAÇÃO DE INPUTS	13
8.6. LOGS	13
8.7. HEALTH CHECKS	13
8.8. REGISTO DE MÉTRICAS	13

8.9. COMUNICAÇÃO SEGURA	13
8.10. ORIGEM DOS DADOS	13
8.11. COMENTÁRIOS NO CÓDIGO	13
8.12. LOCALIZAÇÃO E INTERNACIONALIZAÇÃO	13
8.13. RESPONSABILIDADE	13
8.14. SOFT DELETE	13
8.15. DOCUMENTAÇÃO	14
8.16. TESTES	14
8.16.1. JUnit 5	14
8.16.2. Checkstyle	14
8.16.3. SpotBugs	14

1. ENQUADRAMENTO

O projeto descrito neste documento configura um trabalho a desenvolver pelos formandos da 12ª edição do Programa Acertar o Rumo durante o 3º trimestre da fase letiva. A Critical Software desempenhará o papel de cliente desta solução nomeando um representante que irá apresentar o desafio e esclarecer dúvidas, participando na apresentação final dos trabalhos desenvolvidos. Os alunos ficarão organizados em grupos de 2 elementos.

Os momentos de interação developers/cliente decorrerão de acordo com o seguinte calendário:

- 5 de maio, 14h – apresentação do desafio;
- 12 de maio – data limite para os formandos enviarem as dúvidas por e-mail para o representante do cliente (esousa@criticalsoftware.com) e Prof.ª Naghmeh (naghmeh@dei.uc.pt);
- 19 de maio – data limite para esclarecimento das dúvidas pelo representante do cliente (Elsa Sousa);
- 27 a 29 de maio – apresentação da prototipagem da solução pelos formandos ao cliente no DEI; Sessões individuais entre o representante do cliente e os grupos no DEI;
- 5 de julho – submissão do relatório e término do projeto;
- 6 a 7 de julho – Defesa e demonstração final.

Nos dias 27 a 29 de maio os alunos devem apresentar um conjunto de protótipos rápidos dos “ecrãs” do sistema (e.g., imagens/desenhos) que permitam ao cliente perceber qual será o sistema que os alunos planeiam desenvolver. O objetivo desta sessão é permitir aos alunos perceber se fizeram a interpretação correta dos requisitos propostos pelo cliente e que irão desenvolver o sistema que o cliente espera.

A defesa final será agendada entre os dias 6 a 7 de julho. Para esta defesa os alunos deverão preparar uma demonstração do sistema em funcionamento. Esta demonstração deve estar bem preparada, ou seja, ter um roadmap bem definido que permita aos alunos mostrar que cumpriram todos os requisitos e apresentar as funcionalidades extra que implementaram. Os alunos devem também preparar-se para responder a questões técnicas por parte do júri.

2. AVALIAÇÃO

A avaliação do projeto é baseada na qualidade do **relatório final** e de uma **apresentação oral** efetuada pelo estudante, assim como pela **capacidade do estudante em responder às questões** colocadas pelo Júri. O Júri é nomeado pela Coordenação do Programa Acertar o Rumo, e inclui o presidente, o orientador e restantes membros do júri. A classificação é quantitativa na escala de 0 a 20.

3. GRUPOS E ORIENTADORES

A composição dos grupos de trabalho e os respetivos orientadores são os definidos na seguinte tabela.

Grupo	Aluno 1	Aluno 2	Aluno 3	Orientador
1	Cristina Isabel Conduto da Luz	Joana Sofia Rodrigues Soares	Ana Manuel de Almeida Gonçalves Saraiva Marques	João Ricardo Maximiano Leitão Ribeiro Lourenço
2	José Feliciano Silveira Duarte	Daniel Alexandre Gomes Cação	Dina Maria da Cruz Simões	Marco António Santos Silva
3	Tiago Filipe Correia Pancas	Luís Carlos Ribeiro Ferreira	Francileide Santos de Andrade Vasconcelos	João Miguel Moreira de Carvalho Brás Simões

Tabela 1: Estrutura dos Grupos e Orientadores

4. DOMÍNIO DO PROJETO

4.1. CONTEXTO

A compreensão aprofundada do funcionamento do corpo humano, em condições normais e patológicas, é um fator determinante para a evolução da formação médica, da investigação científica e do treino clínico. A capacidade de observar, analisar e simular reações fisiológicas reais, de forma controlada e reprodutível, é essencial para apoiar a tomada de decisão, a aprendizagem e a validação de intervenções em contextos complexos e de alto risco, na área da medicina.

Contudo, a modelação destes cenários fisiológicos é um processo complexo que exige ferramentas robustas, cientificamente validadas e capazes de representar, de forma integrada, os múltiplos sistemas do corpo humano.

É neste enquadramento que surge a necessidade de desenvolver uma aplicação informática que permita tirar partido de motores de simulação avançados, transformando dados fisiológicos complexos em representações claras, interativas e próximas da realidade clínica.

4.2. OBJETIVOS

O objetivo geral deste projeto é o design e implementação de uma aplicação *web based* que permita a visualização e o acompanhamento da evolução fisiológica de um paciente virtual, de acordo com um cenário fisiológico clínico. A solução deverá implementar as seguintes funcionalidades:

- I. execução da simulação de pelo menos 1 cenário fisiológico clínico de um paciente virtual;
- II. monitorização contínua do estado do paciente, produzindo e apresentando dados sobre os sistemas fisiológicos ao longo do tempo;
- III. destaque visual, numa representação 2D do corpo humano, dos sistemas fisiológicos impactados ao longo do tempo;
- IV. mecanismos de alerta baseados em eventos fisiológicos, de forma a sinalizar alterações relevantes no estado do paciente, facilitando a compreensão da dinâmica da simulação e dos efeitos dos diferentes cenários.

4.3. GLOSSÁRIO DO DOMÍNIO

A Tabela 1 apresenta a lista de conceitos de domínio usados ao longo deste documento.

Conceito	Descrição
Simulação	Processo de execução temporal do modelo fisiológico, durante o qual o motor calcula, de forma contínua e causal, a evolução do estado fisiológico de um paciente virtual.
Cenário fisiológico clínico	Situação fisiológica a ser simulada, que define as condições iniciais e os elementos que irão influenciar a simulação. Exemplos: multi-trauma, golpe de calor, lesão cerebral, etc.
Sistemas fisiológicos	Conjuntos organizados de órgãos que trabalham juntos para realizar funções vitais, garantindo o funcionamento adequado do organismo. Exemplos: cardiovascular, respiratório, renal, nervoso, etc.
Leitura fisiológica	Valor individual ou conjunto de valores que representam medições fisiológicas num determinado instante ou intervalo temporal, incluindo parâmetros como frequência cardíaca, saturação de oxigénio ou pressão arterial.

Conceito	Descrição
Regra	Condição lógica previamente definida, aplicada sobre as leituras fisiológicas para avaliar a ocorrência de situações relevantes ao longo do tempo.
Evento	Ocorrência registada quando uma regra é acionada durante o processamento dos dados fisiológicos.
Alerta	Sinalização gerada pela aplicação, como resultado da ocorrência de um ou mais eventos, indicando que uma condição fisiológica relevante foi detetada.
Avaliação	Processo lógico de análise dos dados fisiológicos ao longo de um determinado período, durante o qual são avaliadas regras, gerados eventos e produzidos alertas, resultando numa interpretação estruturada da evolução fisiológica observada.

Tabela 2: Glossário de Domínio

5. REQUISITOS FUNCIONAIS OBRIGATÓRIOS

Uma função é um 'objetivo definido ou a ação característica de um sistema ou de um componente' e um requisito funcional 'especifica uma função que um sistema ou um componente do sistema devem ser capazes de executar'.

5.1. PERFIS DE UTILIZADOR

O sistema deve suportar a existência de quatro perfis de utilizador (um utilizador está associado a um perfil). A tabela seguinte apresenta a lista dos perfis de utilizador do sistema e as respetivas permissões.

Código de Perfil	Designação	Permissões	Gestão de Utilizador	Gestão de Acessos	Execução de Simulações	Gestão de Alertas	Visualização de Alertas
A	Administrador	Gestão de utilizadores e acessos	✓	✓			
B	Gestor	Execução de simulações e gestão de alertas			✓	✓	✓
C	Utilizador padrão	Execução de simulações e visualização de alertas			✓		✓
D	Utilizador não autenticado	Registo no sistema e recuperação de palavra-passe					

Tabela 3: Perfis de Utilizadores

5.1.1. Perfil de administrador

O administrador deve ser responsável pela configuração global da aplicação, pela gestão de utilizadores e acessos. Permissões principais:

- Criação, edição e remoção de utilizadores;
- Atribuição de perfis e permissões;
- Configurações globais da aplicação que venham a ser necessárias.

5.1.2. Perfil de gestor

O gestor deve conseguir executar simulações, gerir e visualizar alertas.

5.1.3. Perfil de utilizador padrão

O utilizador padrão deve ter apenas acesso à execução de simulações e visualização de alertas.

5.1.4. Perfil de utilizador não autenticado

Um utilizador não autenticado deve ter apenas acesso ao registo e autenticação no sistema, e recuperação de palavra-passe.

5.2. ACESSO E AUTENTICAÇÃO

5.2.1. Registo

O registo de utilizadores deve ser feito inicialmente através do e-mail e de uma palavra-passe (a palavra-passe deve ser solicitada duas vezes para confirmação).

Após o registo, o e-mail deve ser validado através de um link que o utilizador recebe na sua caixa de entrada (do e-mail fornecido no registo). Utilizadores com registo, mas e-mail não validado não podem conseguir autenticar-se no sistema. Após a validação do e-mail, recebem o perfil de Utilizador Padrão.

A palavra-passe deve atender aos requisitos apresentados na secção 6.4.

Nota: Todos os utilizadores devem começar como Utilizador Padrão. Os perfis A e B (secção 5.1) só devem ser atribuídos posteriormente por um administrador de sistema.

5.2.2. Autenticação

Todos os utilizadores devem autenticar-se através do e-mail e palavra-passe. Após uma autenticação bem-sucedida, o sistema deve criar uma sessão que permita ao utilizador ser identificado em pedidos subsequentes, permitindo-lhe exclusivamente acesso às funcionalidades disponíveis para o seu perfil de utilizador.

5.2.3. Terminar Sessão

O utilizador autenticado deve ter, em todos os momentos, acesso a um link que faz a finalização da sessão.

5.2.4. Alterar palavra-passe

Deve existir uma página não autenticada que permita a um utilizador que esqueceu a sua palavra-passe introduzir o seu e-mail e requisitar a alteração da mesma. O utilizador deve receber um link no seu e-mail que permita a alteração da palavra-passe.

A palavra-passe atual não deve ser alterada antes do utilizador clicar no link e submeter uma nova palavra-passe. A palavra-passe deve atender aos requisitos apresentados na secção 6.4.

5.3. EXECUÇÃO DE SIMULAÇÕES

A aplicação deve permitir executar uma simulação de um cenário fisiológico clínico, de um paciente virtual. Ao longo da simulação, deverá ser assegurada a monitorização contínua do estado fisiológico do paciente, com a geração e apresentação de dados relativos aos diferentes sistemas fisiológicos ao longo do tempo. Estes dados deverão ser refletidos através de uma representação bidimensional do corpo humano, na qual os sistemas fisiológicos afetados sejam destacados visualmente de forma dinâmica. Adicionalmente, a aplicação deverá disponibilizar mecanismos de alerta dinâmicos, baseados em eventos fisiológicos, para sinalizar alterações relevantes no estado do paciente ao longo do tempo.

5.3.1. Monitorização Contínua

A aplicação deve assegurar a monitorização contínua do estado fisiológico do paciente virtual ao longo da execução da simulação, recolhendo e processando de forma permanente as leituras fisiológicas recebidas. Esta monitorização deve permitir acompanhar a evolução temporal dos principais sistemas fisiológicos, suportar a avaliação de regras, a deteção de eventos e a atualização dinâmica da representação visual do corpo humano 2D e dos alertas associados.

5.3.2. Representação Visual do Corpo Humano (2D)

A aplicação deve incluir uma representação esquemática bidimensional do corpo humano, com foco nos principais sistemas fisiológicos.

Características da representação:

- Corpo humano apresentado de forma simplificada e intuitiva;
- Sistemas fisiológicos desenhados ou destacados de forma distinta (ex.: respiratório, cardiovascular, renal);

A representação do corpo humano deve ser atualizada em tempo real (ou tempo simulado), refletindo:

- alterações no estado fisiológico,
- degradação ou recuperação de sistemas,
- eventos relevantes ocorridos durante a simulação.

Cada sistema fisiológico representado deve assumir diferentes estados visuais, por exemplo:

- Estado normal – cor neutra;
- Estado de alerta – cor de aviso (ex.: amarelo);
- Estado crítico – cor destacada (ex.: vermelho).

Este mecanismo permite associar imediatamente os valores fisiológicos simulados a uma perceção visual do impacto no corpo humano, sem necessidade de interpretação técnica aprofundada.

5.3.3. Alertas

A geração de alertas ao longo de uma simulação deve acontecer com base em regras e eventos pré-definidos. Para tal, a aplicação deve permitir a configuração de regras com base em:

- limiares fisiológicos;
- variações relativas ou absolutas;
- persistência temporal de determinadas condições;
- combinação entre múltiplos parâmetros.

Quando uma regra é satisfeita, a aplicação deve gerar um alerta associado ao sistema fisiológico afetado.

Os alertas devem ser apresentados de forma clara e contextualizada, incluindo:

- instante temporal;
- evento detectado;
- sistema fisiológico afetado;
- Severidade associada.

Nota: os alertas podem assumir diferentes estados (ativo, resolvido). devem ser atualizados dinamicamente em função da evolução dos dados, e devem refletir-se na interface da aplicação, nomeadamente na representação visual do corpo humano 2D.

A aplicação deve manter um registo cronológico dos alertas envolvidos numa simulação, com detalhe dos mesmos ao longo do tempo.

A aplicação deve ter uma área para gestão de alertas, que permita a criação, edição e eliminação de regras, garantindo que as alterações produzem efeito apenas em avaliações subsequentes, preservando a integridade do histórico. Para além da lógica de visualização do alerta, as regras devem incluir o nível de severidade e o sistema fisiológico afetado, com vista ao mapeamento e à apresentação dos alertas na simulação.

5.4. SUBMISSÃO DE LEITURAS FISIOLÓGICAS E CONSULTA DE ALERTAS

A aplicação deverá disponibilizar endpoints de comunicação que permitam o consumo de leituras fisiológicas e a consulta de alertas.

5.4.1. Submissão de Leituras (Batch)

Disponibilizar endpoints que permitam a submissão de leituras fisiológicas em modo batch, possibilitando o envio de conjuntos de dados em bloco, tipicamente associados a datasets históricos ou a carregamentos periódicos.

5.4.2. Submissão de Leituras (Stream)

Disponibilizar endpoints que permitam a submissão de leituras fisiológicas em modo stream, suportando a receção contínua de dados, de forma sequencial e temporalmente ordenada, simulando ou representando dados em tempo real.

5.4.3. Consulta de Alertas

Disponibilizar endpoints que permitam a consulta dos alertas, correspondentes a eventos fisiológicos atualmente válidos ou persistentes no contexto da execução da simulação ou consumo de dados.

5.4.4. Consulta de Histórico de Alertas

Disponibilizar endpoints que permitam a consulta do histórico de alertas, incluindo todos os alertas gerados ao longo do tempo, independentemente do seu estado atual.

5.5. CARREGAMENTO DE DADOS

É fortemente recomendado que o carregamento de dados seja feito através de uma integração com o BioGears (secção 6.1). Em alternativa, a aplicação pode permitir o carregamento de dados através da leitura de ficheiros, tanto para o cenário fisiológico clínico como para os alertas.

5.5.1. Ficheiro com dados do cenário fisiológico clínico

Exemplo da estrutura do ficheiro json:

```
{  
  "mdibVersion": 42,  
  "device": "PatientMonitor-01",  
  "metrics": [  
    {"handle": "HR", "unit": "1/min", "value": 118, "timestamp": "2026-03-17T10:00:05Z"},  
    {"handle": "SpO2", "unit": "%", "value": 91.5, "timestamp": "2026-03-17T10:00:05Z"}  
  ]  
}
```

5.5.2. Ficheiro com dados de alertas

Exemplo da estrutura do ficheiro yaml:

rules:

```
- id: "NEWS.simple"  
  when:  
    - "hr > 120 for 2m"  
    - "spo2 < 92 for 1m"  
  then: "ALERTA_ALTO"
```

5.6. RELATÓRIO POR AVALIAÇÃO

O relatório por avaliação tem como finalidade assegurar a clareza, rastreabilidade e compreensão dos resultados produzidos pela aplicação, permitindo analisar de forma estruturada os eventos ocorridos e os alertas gerados durante o processamento dos dados fisiológicos. Este relatório fornece uma descrição objetiva das regras acionadas e dos respetivos motivos, baseada nos dados observados, sem constituir qualquer forma de diagnóstico clínico.

Para cada avaliação realizada, a aplicação deve gerar um relatório com informação detalhada e explicativa dos eventos e alertas produzidos durante o processamento dos dados fisiológicos.

Dados que devem constar do relatório:

- **Identificação da avaliação**

Apresentar a identificação da avaliação, incluindo, sempre que aplicável:

- o identificador da sessão ou execução;
- o intervalo temporal considerado;
- o contexto da avaliação (ex.: cenário ou fonte de dados).

- **Listagem de regras acionadas**

Incluir a lista das regras de avaliação que foram efetivamente acionadas durante a avaliação, indicando para cada uma:

- o identificador ou designação da regra;
- o sistema fisiológico associado;
- o instante ou intervalo temporal em que a regra foi acionada;
- o tipo de evento ou alerta resultante.

- **Rationale legível**

Associar a cada regra acionada um rationale em linguagem clara e legível, que explique o motivo da sua ativação com base nos dados observados, sem recorrer a terminologia diagnóstica ou inferência clínica.

- **Consulta do Relatório**

Disponibilizar o relatório por avaliação para consulta posterior, permitindo a sua utilização em contextos de análise, revisão e validação.

6. REQUISITOS FUNCIONAIS OPCIONAIS (BÓNUS)

6.1. MODO DEGRADADO (FORTEMENTE RECOMENDADO)

A aplicação deve implementar um modo degradado que permita continuar a avaliação e geração de alertas em memória, sempre que a base de dados esteja indisponível, assegurando a posterior sincronização automática dos dados, após a recuperação da persistência.

Quando ocorrer uma falha de acesso, escrita ou leitura na base de dados, a aplicação deve:

- Manter em memória toda a informação necessária à avaliação em curso, incluindo:
 - o leituras fisiológicas recebidas,
 - o estados dos sistemas fisiológicos,
 - o alertas gerados e respetivos metadados;
- Continuar a lógica de avaliação e geração de alertas, sem interrupção do funcionamento funcional da aplicação;
- Sinalizar a condição de operação em modo degradado, de forma transparente, para efeitos de monitorização e diagnóstico.

Após a recuperação da base de dados, a aplicação deve:

- Restabelecer automaticamente a ligação à base de dados;
- Sincronizar os dados mantidos em memória, garantindo a sua persistência sem perda ou duplicação de informação;
- Preservar a ordem temporal e a integridade das leituras e dos alertas armazenados durante o período de falha;
- Retomar o funcionamento normal sem necessidade de intervenção manual.

Este mecanismo deve ser implementado de forma a minimizar perda de dados, assegurar a consistência do sistema e aumentar a resiliência da aplicação face a falhas temporárias da infraestrutura de persistência.

6.2. INTEGRAÇÃO COM O BIOGEARS (FORTEMENTE RECOMENDADO)

O BioGears é um motor de simulação fisiológica humana de código aberto, validado cientificamente, orientado para educação médica, investigação biomédica e treino clínico. Permite a execução de simulações de um paciente virtual em tempo real, com base em parâmetros fisiológicos configuráveis. O BioGears permite:

- simular a fisiologia humana de forma integrada e causal;
- produzir dados contínuos sobre sistemas fisiológicos (cardiovascular, respiratório, renal, nervoso, entre outros);
- gerir a evolução temporal de cenários previamente configurados.

A aplicação deve consumir os dados do BioGears, não alterando o seu funcionamento interno, mas utilizando-os como fonte de informação fisiológica fidedigna de um paciente virtual.

Está disponível através do site biogearsengine.com.

6.3. INTEROPERABILIDADE

O IEEE 11073 SDC (Service-oriented Device Connectivity) é um standard de interoperabilidade para dispositivos médicos reais, e foi criado para resolver um problema central na área da saúde:

- permitir que diferentes sistemas e dispositivos médicos comuniquem entre si de forma normalizada, segura e sem dependências proprietárias.

Assim sendo, em contexto de dispositivos médicos, monitorização e sistemas clínicos, o IEEE 11073 SDC deve ser usado quando o objetivo da aplicação for consumir, normalizar e integrar dados fisiológicos de forma interoperável, escalável e independente da origem.

Para garantir a interoperabilidade entre diferentes sistemas e dispositivos médicos, a aplicação deve disponibilizar endpoints que permitam o consumo de leituras fisiológicas através de mecanismos SDC-inspired (IEEE 11073 SDC), garantindo a normalização semântica dos dados e a independência relativamente à sua origem (motor de simulação, dataset ou fonte externa).

6.4. FEATURE FLAGS

Feature flags ou feature toggles permitem a ativação ou desativação de funcionalidades específicas, em tempo real, sem necessidade de se proceder a uma nova instalação da aplicação. Apenas administradores do sistema devem ter permissões para gerir feature flags.

7. REQUISITOS TÉCNICOS

A aplicação deve ser desenvolvida tendo em consideração os seguintes requisitos técnicos:

- Java 17+, Maven;
- API REST (Spring Boot/JAXRS) com modos batch (CSV/JSON) e stream (POST incremental);
- Motor de regras com miniDSL YAML/JSON (limiares + janelas temporais);
- WebSockets para notificações/eventos em tempo real;
- Persistência leve (H2/SQLite) + cache inmemory;
- Frontend: ReactJS + Zustand.

8. REQUISITOS DE QUALIDADE E SEGURANÇA

Os programadores devem aderir às boas práticas de segurança e qualidade de software. Os requisitos que se seguem visam garantir a segurança dos sistemas contra ameaças à confidencialidade, integridade e disponibilidade.

8.1. SESSION TIMEOUT

Um utilizador autenticado tem de ter a sua sessão terminada após um período de inatividade, configurável pelo administrador do sistema.

8.2. FORÇA DAS PALAVRAS-PASSE

As palavras-passe devem ser validadas de acordo com as melhores práticas de segurança da informação, e não deve ser possível que os utilizadores selecionem palavras-passe fracas.

8.3. ARMAZENAMENTO DE PALAVRAS-PASSE

As palavras-passe não podem ser armazenadas de forma legível na base de dados, devendo ser utilizado um método seguro de armazenamento.

8.4. CONTROLO DE ACESSO

O controlo de acesso da aplicação não deve depender de informações que sejam armazenadas ou recebidas pelo browser/cliente, devendo depender apenas de informações associadas à sessão e armazenadas no servidor.

8.5. VALIDAÇÃO DE INPUTS

Todos os inputs de utilizadores devem ser validados e filtrados para evitar ataques comuns como *Injeção de SQL*, *Cross Site Scripting*.

8.6. LOGS

Todas as atividades executadas no sistema, incluindo operações de administradores, devem ficar registadas em logs protegidos, com data, hora, autor e IP de origem.

Os logs devem permitir o registo de eventos relevantes como o consumo de dados, avaliação de regras, geração de alertas, e a identificação de falhas ou do início do modo degradado.

8.7. HEALTH CHECKS

A aplicação deve disponibilizar health checks, ou seja, mecanismos que permitem verificar automaticamente o seu estado de funcionamento.

8.8. REGISTO DE MÉTRICAS

A aplicação deve expor métricas operacionais através do Micrometer, para monitorizar o desempenho e a carga da aplicação, identificar bottle necks ou comportamentos anómalos, e avaliar a estabilidade e escalabilidade ao longo do tempo.

8.9. COMUNICAÇÃO SEGURA

A aplicação deve comunicar apenas através do protocolo seguro HTTPS.

8.10. ORIGEM DOS DADOS

Todos os datasets devem ser 100% sintéticos, sem referências a clientes ou projetos reais.

8.11. COMENTÁRIOS NO CÓDIGO

O código deve estar bem comentado, facilitando a manutenção.

8.12. LOCALIZAÇÃO E INTERNACIONALIZAÇÃO

A aplicação deve suportar pelo menos dois idiomas.

8.13. RESPONSABILIDADE

A aplicação deve ser responsiva para telemóveis, tablets e computadores de secretária.

8.14. SOFT DELETE

Quando registros são excluídos, a prática recomendada não deve ser removê-los fisicamente, mas sim marcá-los como "inativos" ou "excluídos". Em certos casos, pode ser fornecida a opção de restaurar esses dados inativos ou excluí-los permanentemente.

8.15. DOCUMENTAÇÃO

Os programadores devem documentar o código utilizando Javadoc.

8.16. TESTES

Toda a aplicação, tanto o backend quanto o frontend, deve ser amplamente testada, incluindo testes unitários, testes de API, testes de integração e testes de frontend.

8.16.1. JUnit 5

Pelo menos 70% do código da camada de domínio deve estar coberto por testes unitários, com recurso ao JUnit 5.

8.16.2. Checkstyle

O código deverá ser automaticamente analisado, com recurso a Checkstyle, para verificar o cumprimento de convenções de nomenclatura, formatação e indentação, estrutura de classes e métodos, e boas práticas de legibilidade.

8.16.3. SpotBugs

Deve ser feita uma análise estática do código, com recurso a SpotBugs, para identificar:

- potenciais defeitos;
- erros lógicos comuns;
- más práticas de programação;
- problemas de segurança ou concorrência.